

HOSYS - Hotel Billing & Restaurant Management System

Phase 1 Documentation

1. Project Overview

HOSYS is a Hotel Billing and Restaurant Management System built using Spring Boot, PostgreSQL, JPA/Hibernate, and REST APIs. The goal is to manage restaurant tables, menu items, orders, billing, payments, and user roles through a centralized backend.

2. Requirement Analysis

Identified major modules: User Management, Role Management, Restaurant Tables, Menu Categories, Menu Items, Orders, Order Items, Bills, and Payments. Defined relationships and business flow before implementation.

3. ER Diagram Design

Designed an ER Diagram containing: Roles, Users, Restaurant Tables, Menu Categories, Menu Items, Orders, Order Items, Bills, and Payments. Relationships such as One-to-Many and Many-to-One were defined according to restaurant operations.

4. Database Schema Design

Created schema.sql based on the ER Diagram. Defined primary keys, foreign keys, constraints, and relationships. PostgreSQL database 'hosys_db' was created and connected to the application.

5. Spring Boot Project Setup

Generated Spring Boot project using Spring Initializr. Dependencies: - Spring Web - Spring Data JPA - PostgreSQL Driver - Spring Security - Lombok - Validation

6. PostgreSQL Integration

Configured application.properties with PostgreSQL connection details. Successfully established JDBC connection. Verified Hibernate startup and schema synchronization.

7. Project Structure

Created layered architecture: config/ controller/ dto/ entity/ exception/ repository/ security/ service/ util/

8. Entity Creation

Implemented JPA Entities: Role User RestaurantTable MenuCategory MenuItem Order OrderItem Bill Payment Added entity relationships using @ManyToOne and @JoinColumn.

9. Repository Layer

Created JpaRepository interfaces for entities. Enabled database CRUD operations through Spring Data JPA.

10. Role Seeder

Implemented RoleSeeder using CommandLineRunner. Automatically inserts: ADMIN MANAGER WAITER CHEF on application startup if roles do not already exist.

11. Service Layer

Created UserService. Implemented: - getAllUsers() - saveUser() Business logic is separated from controller logic.

12. REST API Development

Created UserController. Endpoints: GET /api/users POST /api/users Successfully tested APIs using Thunder Client.

13. API Testing Results

User records were successfully inserted into PostgreSQL. GET requests returned stored users. User-role relationship was verified through API responses.

14. Issues Solved During Development

- Maven not recognized globally. - PostgreSQL connection configuration. - Driver class detection errors. - Port 8080 conflicts. - Entity relationship mapping issues. - Role seeding and repository validation.

15. Current Project Status

Completed: ✓ ER Diagram ✓ Database Schema ✓ PostgreSQL Integration ✓ Spring Boot Setup ✓ Entity Layer ✓ Repository Layer ✓ User Service ✓ User Controller ✓ Role Seeder ✓ API Testing
Next: MenuCategory Module MenuItem Module Restaurant Table Module Order Management Billing Module Payment Module Security & Authentication Frontend Integration